



Docker Storage

Persistent & Non-Persistent Storage, Bind Mounts & Named Volumes

A complete step-by-step tutorial with a live Node.js demo application.
Learn when to use each storage strategy and how to apply it in practice.

Table of Contents

1. Introduction
2. How Docker Handles Storage — The Layered Filesystem
3. Non-Persistent Storage — Container Layer
4. Persistent Storage — Overview
5. The Demo Application — Visitor Log App
6. Building the Docker Image
7. Demo A — Non-Persistent Storage (Baseline)
8. Bind Mounts — Deep Dive
9. Demo B — Bind Mount Step-by-Step
10. Docker Volumes — Deep Dive
11. Demo C — Named Volume Step-by-Step
12. Bind Mounts vs Docker Volumes — Full Comparison
13. When to Use Which Storage Type
14. Advanced: Read-Only Mounts & tmpfs
15. Troubleshooting Common Storage Issues
16. Quick-Reference Cheat Sheet
17. Summary

1. Introduction

Docker containers are ephemeral by design — when a container is removed, everything written inside it disappears. This is intentional: it keeps containers stateless, portable, and reproducible. However, real-world applications almost always need to persist data — log files, databases, user uploads, configuration, and more.

This tutorial explores Docker's three storage options in full depth:

Storage Type	Managed By	Persists After rm?	Best For
Container Layer	Docker (ephemeral)	X No	Throwaway scratch data
Bind Mount	Host OS	✓ Yes	Dev workflows, config files
Named Volume	Docker Engine	✓ Yes	Production databases, logs

By the end of this tutorial you will be able to:

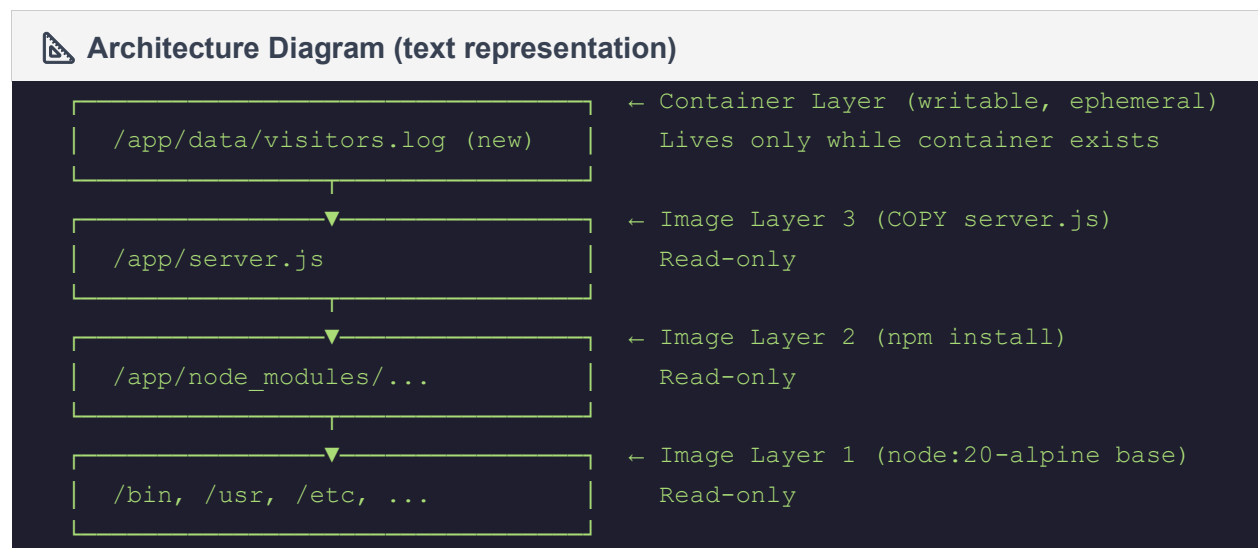
- Explain the Docker layered filesystem and why data is lost by default.
- Mount a host directory into a container using a Bind Mount.
- Create and manage Named Volumes for production-grade persistence.
- Choose the right storage strategy for each use case.

2. How Docker Handles Storage — The Layered Filesystem

Every Docker image is built from a stack of read-only layers. When you run a container, Docker adds a thin, writable layer on top called the Container Layer (or writable layer). All changes made inside the running container — new files, modified files, deleted files — are written to this layer only.

2.1 The Union File System (OverlayFS)

Docker uses a Union File System (most commonly OverlayFS on Linux) to merge all image layers plus the writable container layer into a single, unified view. From inside the container it looks like one flat filesystem, but Docker tracks changes at the layer level.



2.2 Why Data Is Lost

When you run `docker rm <container>`, Docker deletes the Container Layer entirely. All files written during the container's lifetime are gone. The image layers below are untouched and can create a fresh writable layer for the next container.

 **Key Insight** Stopping a container does NOT delete its writable layer. Data survives a stop/start cycle. Only `docker rm` (or `docker run --rm`) destroys the layer.

3. Non-Persistent Storage — Container Layer

3.1 Characteristics

- Writes go directly to the container's writable layer.
- Fast — no extra overhead for mounting external storage.
- Completely isolated — other containers cannot share this data.
- **Data is destroyed** when the container is removed (`docker rm`).

3.2 When Is It Acceptable?

- Throwaway computation (e.g., compiling an asset, running tests).
- Caching that should reset on every container restart.
- Data that is always regenerated by the application on startup.

3.3 When Is It a Problem?

- Log files that must survive restarts for auditing or debugging.
- Databases — all records would be wiped on every restart.
- User-uploaded files, machine-learning model checkpoints, session data.

 **Rule of Thumb** If losing the data would cause a bug, business problem, or data-loss incident — you need persistent storage.

4. Persistent Storage — Overview

Docker provides two main mechanisms to persist data outside the container layer:

Feature	Bind Mount	Named Volume
Storage location	Exact path on host OS	Docker-managed directory (usually /var/lib/docker/volumes/)
Who controls path?	Developer / Operator	Docker Engine
Portability	Tied to host directory structure	Portable across hosts with volume drivers
Initial content	Shows host directory as-is (can be empty)	Pre-populated from image if volume is new
Cross-platform	Path differences on Win/Mac/Linux	Consistent on all platforms
Performance	Slight overhead on Mac/Windows (VM)	Best performance on Linux
Backup / migrate	Standard OS file tools	docker volume + container-based backup
Access without Docker	Yes — direct filesystem access	Requires running a container

Both mechanisms mount external storage at a specific path inside the container. From the application's point of view, it simply reads and writes files at that path — it does not know whether a bind mount or a volume is backing it.

5. The Demo Application — Visitor Log App

To make the storage concepts tangible we use a purposefully storage-dependent application: a Visitor Log app built with Node.js and Express.

5.1 What the App Does

- Displays a web form where users enter their name.
- On submit, appends a timestamped entry to `/app/data/visitors.log` inside the container.
- Reads and displays all log entries on the homepage.
- Provides a clear button to reset the log (demo convenience).

🔗 **Why This App?** The log file is the perfect demo artifact: you can add entries, remove the container, restart it, and immediately see whether the data survived — making storage behaviour 100% visible.

5.2 Project Structure

```
docker-storage-demo/  
├─ Dockerfile  
├─ .dockerignore  
└─ app/  
    ├─ server.js      ← Node.js + Express application  
    └─ package.json   ← Dependencies (express)
```

5.3 The Dockerfile

```
# _____  
# Dockerfile - Visitor Log App  
# _____  
  
FROM node:20-alpine  
  
# Set working directory  
WORKDIR /app  
  
# Install dependencies (layer cache optimisation)  
COPY package*.json ./  
RUN npm install --omit=dev  
  
# Copy application source
```

```
COPY server.js ./

# Create the data directory (overridden by mount at runtime)
RUN mkdir -p /app/data

# Document the volume mount point
VOLUME ["/app/data"]

# Expose app port
EXPOSE 3000

# Non-root user (security best practice)
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
RUN chown -R appuser:appgroup /app
USER appuser

CMD ["node", "server.js"]
```

5.4 Key Application Code — Writing to the Log File

The relevant server.js snippet that writes to disk:

```
const LOG_DIR = path.join(__dirname, 'data');
const LOG_FILE = path.join(LOG_DIR, 'visitors.log');

// Append a new visitor entry
app.post('/sign', (req, res) => {
  const ts = new Date().toISOString().slice(0, 19).replace('T', ' ');
  const entry = `${ts} | ${req.body.name}\n`;
  fs.appendFileSync(LOG_FILE, entry);
  res.redirect('/');
});
```


6. Building the Docker Image

6.1 Navigate to the Project Directory

```
# macOS / Linux
cd ~/docker-storage-demo

# Windows (PowerShell)
cd C:\Users\YourName\docker-storage-demo
```

6.2 Build the Image

```
docker build -t visitor-log-app:v1 .
```

6.3 Expected Output

```
[+] Building 18.3s (11/11) FINISHED
=> [1/6] FROM docker.io/library/node:20-alpine
=> [2/6] WORKDIR /app
=> [3/6] COPY package*.json ./
=> [4/6] RUN npm install --omit=dev
=> [5/6] COPY server.js ./
=> [6/6] RUN mkdir -p /app/data
=> exporting to image
=> naming to docker.io/library/visitor-log-app:v1 DONE
```

6.4 Verify the Image

```
docker images visitor-log-app

# Expected:
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
visitor-log-app	v1	a1b2c3d4e5f6	2 minutes ago	131MB

7. Demo A — Non-Persistent Storage (Baseline)

Before using any persistent storage, run the app without any mount to observe data loss firsthand.

Step 1 — Start the container (no mount)

```
docker run -d -p 3000:3000 --name visitor-app visitor-log-app:v1
```

Step 2 — Open the app and add visitor entries

Navigate to <http://localhost:3000> and submit 2–3 visitor names. Observe that entries appear in the log table.

Step 3 — Stop and REMOVE the container

```
docker stop visitor-app  
docker rm visitor-app
```

Step 4 — Start a fresh container

```
docker run -d -p 3000:3000 --name visitor-app visitor-log-app:v1
```

Step 5 — Check the app

Open <http://localhost:3000> again. The visitor log is empty — all entries are gone.

 **Observation** The log file lived in the writable Container Layer. `docker rm` deleted that layer. The image is untouched — only the runtime data was lost. This is the core problem that Bind Mounts and Volumes solve.

Step 6 — Cleanup

```
docker stop visitor-app && docker rm visitor-app
```

8. Bind Mounts — Deep Dive

8.1 What Is a Bind Mount?

A Bind Mount maps a specific directory (or even a single file) on the host machine's filesystem directly into the container. The container and the host share the exact same bytes — it is not a copy, it is a live two-way link.

Host Filesystem		Container Filesystem
<code>~/docker-storage-demo/</code>		<code>/app/</code>
└─ <code>data/</code>	↔	└─ <code>data/</code>
└─ <code>visitors.log</code>		└─ <code>visitors.log</code>

Changes on the host are instantly visible inside the container.
Changes inside the container are instantly visible on the host.

8.2 The -v Flag Syntax

```
docker run -v <HOST_PATH>:<CONTAINER_PATH>[:<OPTIONS>] ...

# Example:
docker run -v /home/user/data:/app/data visitor-log-app:v1

# With read-only option:
docker run -v /home/user/config:/app/config:ro visitor-log-app:v1

# Modern --mount syntax (more explicit):
docker run --mount type=bind,source=/home/user/data,target=/app/data visitor-
log-app:v1
```

8.3 Bind Mount Characteristics

- **Live sync:** Any change on the host is immediately visible inside the container and vice versa. No restart required.
- **Host path must exist:** Docker will create the host path if it does not exist, but this can lead to permission issues. It is best practice to pre-create the directory.
- **Overwrites container content:** If the container image has files at the mount point, they are hidden (shadowed) by the bind mount. An empty host directory makes the container see an empty directory.
- **Permission sensitive:** The container process must have OS-level read/write access to the host path.

8.4 Trade-offs of Bind Mounts

Advantage	Disadvantage
Simple to set up — just specify a path	Host path must be created manually
Direct file access on the host (cat, tail, IDE)	Tightly couples container to host directory structure
Perfect for live-reload development workflows	Path differs between OS (Windows vs Linux vs Mac)
Easy to inspect and edit config files in place	Security risk: container can write anywhere on host
Supports single-file mounts (e.g., nginx.conf)	Not suitable for sharing data between multiple containers

8.5 Primary Use Cases for Bind Mounts

- Development: mount source code so changes reflect inside container without rebuild.
- Configuration injection: mount a config file (e.g., nginx.conf, app.yml) into the container.
- Log collection: expose container logs directly to host monitoring tools.
- Sharing data between the host and a single container.

9. Demo B — Bind Mount Step-by-Step

Step 1 — Create the host data directory

Create a folder on your host that will be mounted into the container.

```
# macOS / Linux
mkdir -p ~/docker-storage-demo/data

# Windows (PowerShell)
New-Item -ItemType Directory -Path "$env:USERPROFILE\docker-storage-demo\data"
-Force
```

Step 2 — Run the container WITH a Bind Mount

```
# macOS / Linux
docker run -d -p 3000:3000 --name visitor-app \
  -v ~/docker-storage-demo/data:/app/data \
  visitor-log-app:v1

# Windows (PowerShell) - use $PWD for current directory
docker run -d -p 3000:3000 --name visitor-app `
  -v "${env:USERPROFILE}\docker-storage-demo\data:/app/data" `
  visitor-log-app:v1

# Using --mount syntax (cross-platform recommended):
docker run -d -p 3000:3000 --name visitor-app \
  --mount type=bind,source=$(pwd)/data,target=/app/data \
  visitor-log-app:v1
```

 **\$(pwd) vs absolute path** Always use absolute paths with bind mounts. \$(pwd) expands to the current working directory on Linux/Mac. On Windows use \$PWD in PowerShell.

Step 3 — Add visitor entries

Open <http://localhost:3000> and add 3–4 visitor names.

Step 4 — Verify the log file on the HOST

Open a new terminal and inspect the host directory:

```
# Linux / macOS
```

```
cat ~/docker-storage-demo/data/visitors.log

# Expected output:
2025-06-10 14:23:01 | Alice
2025-06-10 14:23:15 | Bob
2025-06-10 14:23:28 | Carol

# You can also tail it live:
tail -f ~/docker-storage-demo/data/visitors.log
```

 **Key Observation** The log file is on YOUR host machine right now. You can open it with any text editor, cat it, grep it — without touching Docker at all.

Step 5 — Stop and REMOVE the container

```
docker stop visitor-app
docker rm visitor-app
```

Step 6 — Verify data survived on the host

```
cat ~/docker-storage-demo/data/visitors.log

# Still there:
2025-06-10 14:23:01 | Alice
2025-06-10 14:23:15 | Bob
2025-06-10 14:23:28 | Carol
```

Step 7 — Restart a fresh container pointing to the same data

```
docker run -d -p 3000:3000 --name visitor-app \
-v ~/docker-storage-demo/data:/app/data \
visitor-log-app:v1
```

Open <http://localhost:3000> — your previous log entries are shown! The data persisted through the container lifecycle.

Step 8 — Test two-way sync (write from host, read in container)

```
# Add an entry directly from the host (without using the web form)
echo "$(date +%Y-%m-%d %H:%M:%S) | ManualHostEntry" >> ~/docker-storage-
demo/data/visitors.log

# Refresh http://localhost:3000 — the new entry appears instantly
```

Step 9 — Cleanup

```
docker stop visitor-app && docker rm visitor-app
```

10. Docker Volumes — Deep Dive

10.1 What Is a Named Volume?

A Named Volume is a storage unit fully managed by the Docker Engine. You give it a name; Docker handles its location on disk (typically `/var/lib/docker/volumes/<name>/_data` on Linux). Unlike bind mounts, you never deal with host filesystem paths.

```
Docker Engine manages:
/var/lib/docker/volumes/
└─ visitor_data/
    └─ _data/
        └─ visitors.log ← your app data

Inside the container, mounted at:
/app/data/visitors.log
```

10.2 Volume Lifecycle Commands

```
# Create a volume explicitly
docker volume create visitor_data

# List all volumes
docker volume ls

# Inspect a volume (see its mount point on host)
docker volume inspect visitor_data

# Remove a volume
docker volume rm visitor_data

# Remove ALL unused volumes (prune)
docker volume prune
```

10.3 How Volumes Differ from Bind Mounts at Startup

An important but often-missed behaviour: when a Named Volume is new (empty) and is mounted over a container path that already contains files in the image, Docker copies those files from the image into the volume. Bind mounts do NOT do this — they shadow the container content entirely.

Scenario	Bind Mount	Named Volume
----------	------------	--------------

Mount path has files in image, volume/host dir is empty	Container sees EMPTY directory (host dir shadows image content)	Container sees image files (Docker seeds the volume on first use)
Mount path has files in image, volume/host dir has files	Container sees ONLY host files	Container sees ONLY volume files

10.4 Volume Drivers

The default volume driver is local (stores data on the Docker host). Production deployments commonly use third-party drivers to store volumes remotely:

- **amazon-efs-volumes** — Store volumes on AWS Elastic File System
- **azure-file-storage** — Azure Files share
- **rexray/ebs** — AWS EBS block storage
- **convoy** — Pluggable block storage with snapshot support

10.5 Trade-offs of Named Volumes

Advantage	Disadvantage
Fully managed — Docker handles path & permissions	Cannot directly edit files with host text editors
Portable: same volume name works on any Docker host	Requires docker exec or helper container to inspect content
Best performance on Linux (native kernel)	Slightly harder to set up cross-container backup pipelines
Secure: isolated from rest of host filesystem	Volumes persist until explicitly removed (can accumulate)
Seeds data from image on first use	Not ideal for live code reload in development
Works seamlessly with Docker Compose and Swarm	

11. Demo C — Named Volume Step-by-Step

Step 1 — Create the Named Volume

You can create a volume explicitly, or let Docker create it automatically on first use.

```
# Explicit creation (recommended — makes intent clear):
docker volume create visitor_data

# Verify it was created:
docker volume ls

# Expected:
DRIVER      VOLUME NAME
local       visitor_data
```

Step 2 — Inspect the Volume

```
docker volume inspect visitor_data

# Output:
[
  {
    "CreatedAt": "2025-06-10T14:30:00Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/visitor_data/_data",
    "Name": "visitor_data",
    "Options": {},
    "Scope": "local"
  }
]
```

 **Note** The Mountpoint is the actual location on the host. On Linux you can cd there directly. On Mac/Windows it lives inside the Docker Desktop VM and is not directly accessible.

Step 3 — Run the container using the Named Volume

```
docker run -d -p 3000:3000 --name visitor-app \
  -v visitor_data:/app/data \
  visitor-log-app:v1
```

```
# --mount syntax equivalent:
docker run -d -p 3000:3000 --name visitor-app \
  --mount type=volume,source=visitor_data,target=/app/data \
  visitor-log-app:v1
```

Step 4 — Add visitor entries

Open <http://localhost:3000> and sign 3–4 visitor names.

Step 5 — Verify volume contains the data

```
# Read the log file from inside the volume using a helper container:
docker run --rm -v visitor_data:/data alpine cat /data/visitors.log

# Expected:
2025-06-10 14:35:01 | Alice
2025-06-10 14:35:18 | Bob
2025-06-10 14:35:34 | Carol
```

Step 6 — Stop and REMOVE the container

```
docker stop visitor-app
docker rm  visitor-app

# Confirm the volume still exists:
docker volume ls

# visitor_data is still listed — it outlives the container
```

Step 7 — Re-attach a new container to the same volume

```
docker run -d -p 3000:3000 --name visitor-app \
  -v visitor_data:/app/data \
  visitor-log-app:v1
```

Open <http://localhost:3000> — previous entries are fully restored. The Named Volume persisted the data independently of the container.

Step 8 — Share data between two containers (volume reuse)

Demonstrate that two containers can mount the same volume simultaneously:

```
# Start a second container on a different port:
docker run -d -p 3001:3000 --name visitor-app-2 \
  -v visitor_data:/app/data \
  visitor-log-app:v1
```

```
# Open http://localhost:3001 – it shows the same log entries!
# Add an entry in one app, refresh the other – they share the same data.
```

Step 9 — Backup the volume

```
# Standard backup pattern: run a temporary alpine container,
# mount the volume + a host backup dir, tar the data.
docker run --rm \
  -v visitor_data:/source:ro \
  -v $(pwd)/backups:/backup \
  alpine tar czf /backup/visitor_data_backup.tar.gz -C /source .

# Restore from backup:
docker run --rm \
  -v visitor_data:/target \
  -v $(pwd)/backups:/backup \
  alpine tar xzf /backup/visitor_data_backup.tar.gz -C /target
```

Step 10 — Cleanup

```
docker stop visitor-app visitor-app-2
docker rm  visitor-app visitor-app-2

# Remove the volume (WARNING: deletes all data in it):
docker volume rm visitor_data
```

12. Bind Mounts vs Docker Volumes — Full Comparison

Aspect	Container Layer	Bind Mount	Named Volume
Data persistence	✗ Lost on rm	✓ Survives rm	✓ Survives rm
Storage location	Container writable layer	Exact host path	Docker-managed path
Path control	None	Full (developer)	Docker handles it
Host file access	✗	✓ Direct	✗ (via container)
Cross-container sharing	✗	Awkward (path-based)	✓ Native
Performance (Linux)	Best	Very good	Best
Performance (Mac/Win)	Best	Slower (VM overhead)	Faster than bind
Dev live-reload	✗	✓ Ideal	Possible but awkward
Production DB/logs	✗ Never	Workable	✓ Recommended
Backup mechanism	N/A	OS file tools	docker run + tar
Docker Compose support	N/A	✓	✓
Security isolation	Container boundary	Host path exposed	Fully isolated

13. When to Use Which Storage Type

Use Container Layer when:

- Data is temporary and intentionally ephemeral (build artefacts, temp files).
- The container is a one-shot task that does not produce persistent output.
- You want maximum startup speed and zero external dependencies.

Use Bind Mounts when:

- Developing locally — mount source code so edits reflect instantly without rebuild.
- Injecting configuration files from the host (e.g., nginx.conf, .env files).
- Collecting logs into a known host location for external log shippers.
- Running data-processing scripts that read/write files managed by the host.

 **Avoid bind mounts in production for application data** They create a strong coupling to the host directory structure and can expose sensitive host paths to the container.

Use Named Volumes when:

- Running databases (PostgreSQL, MySQL, MongoDB) — the gold standard.
- Persisting application logs in multi-container or orchestrated environments.
- Sharing data between multiple containers (e.g., app + sidecar log forwarder).
- Deploying to Kubernetes/Swarm where host paths differ between nodes.
- You want Docker to manage backup, lifecycle, and permissions.

14. Advanced: Read-Only Mounts & tmpfs

14.1 Read-Only Bind Mount

Adding `:ro` prevents the container from modifying the mounted files — useful for injecting config the container should read but not modify.

```
# Bind mount in read-only mode
docker run -v $(pwd)/config:/app/config:ro visitor-log-app:v1

# --mount syntax:
docker run --mount type=bind,source=$(pwd)/config,target=/app/config,readonly
visitor-log-app:v1
```

14.2 Read-Only Named Volume

```
docker run -v visitor_data:/app/data:ro visitor-log-app:v1

# --mount syntax:
docker run --mount type=volume,source=visitor_data,target=/app/data,readonly
visitor-log-app:v1
```

14.3 tmpfs Mount (In-Memory, Non-Persistent)

A tmpfs mount stores data in the host's RAM — never written to disk. Data is lost when the container stops. Useful for sensitive data (secrets, session tokens) that must not be persisted.

```
docker run --tmpfs /app/tmp:rw,size=64m visitor-log-app:v1

# --mount syntax:
docker run --mount type=tmpfs,target=/app/tmp,tmpfs-size=67108864 visitor-log-
app:v1
```

Mount Type	Persists?	Accessible on Host?	Use Case
Bind (rw)	✓	✓	Dev workflow, config injection
Bind (ro)	✓	✓	Read-only config / certs
Volume (rw)	✓	✗	Production data storage
Volume (ro)	✓	✗	Shared read-only assets
tmpfs	✗	✗	Sensitive in-memory data

15. Troubleshooting Common Storage Issues

Problem	Likely Cause	Solution
Container sees empty directory even though host has files	Path typo in -v flag, or relative path used	Use absolute path: \$(pwd)/data instead of ./data
Permission denied writing to bind mount	Container user UID does not match host directory owner	Run: <code>chmod 777 ./data</code> or <code>chown <uid>:<gid> ./data</code> on host
Data not persisting after docker stop (not rm)	Normal — stop preserves the writable layer	Use <code>docker start</code> to resume. Only <code>docker rm</code> destroys the layer.
Data not persisting after docker rm	No mount was used — data was in container layer	Add -v flag to docker run: <code>-v visitor_data:/app/data</code>
Volume exists but app sees old data	Wrong volume name — check <code>docker volume ls</code>	Run <code>docker volume inspect <name></code> to verify mount point contents
Cannot access volume files from host (Mac/Windows)	Volume lives inside Docker Desktop VM	Use helper container: <code>docker run --rm -v vol:/data alpine ls /data</code>
<code>docker volume rm</code> fails with "volume is in use"	A container (even stopped) references the volume	Run <code>docker rm <container></code> first, then <code>docker volume rm <vol></code>
bind mount slower than expected on Mac/Windows	Performance overhead of host↔VM filesystem bridge	Use Named Volume instead, or enable VirtioFS in Docker Desktop settings

16. Quick-Reference Cheat Sheet

Container Layer (no mount)

```
docker run -d -p 3000:3000 --name myapp visitor-log-app:v1
# Data is lost when container is removed
```

Bind Mount

```
# -v syntax
docker run -d -p 3000:3000 --name myapp \
  -v /absolute/host/path:/app/data \
  visitor-log-app:v1

# --mount syntax (recommended)
docker run -d -p 3000:3000 --name myapp \
  --mount type=bind,source=$(pwd)/data,target=/app/data \
  visitor-log-app:v1

# Read-only bind mount
docker run -v $(pwd)/config:/app/config:ro visitor-log-app:v1
```

Named Volume

```
# Create volume
docker volume create mydata

# Run with named volume
docker run -d -p 3000:3000 --name myapp \
  -v mydata:/app/data \
  visitor-log-app:v1

# Inspect / list / remove volumes
docker volume ls
docker volume inspect mydata
docker volume rm mydata
docker volume prune          # remove all unused volumes

# Read volume contents without running the main app
docker run --rm -v mydata:/data alpine ls /data

# Backup a volume
docker run --rm -v mydata:/src:ro -v $(pwd):/bak alpine \
  tar czf /bak/backup.tar.gz -C /src .
```

```
# Restore a volume
docker run --rm -v mydata:/tgt -v $(pwd):/bak alpine \
  tar xzf /bak/backup.tar.gz -C /tgt
```

17. Summary

This tutorial covered Docker's complete storage model from first principles to practical production patterns:

1. Learned that containers write to an ephemeral writable layer deleted on `docker rm`.
2. Observed data loss firsthand in Demo A using no mount.
3. Mastered Bind Mounts — two-way live link between host path and container.
4. Executed Demo B: persisted the visitor log on the host, verified host-side access.
5. Mastered Named Volumes — Docker-managed storage with full lifecycle control.
6. Executed Demo C: persisted across container removal, shared between two containers, performed a backup/restore.
7. Compared all three storage types across performance, use cases, and trade-offs.
8. Explored read-only mounts and `tmpfs` for advanced security patterns.

 **Next Steps** Explore Docker Compose volumes (the `volumes`: top-level key), Kubernetes `PersistentVolumeClaims`, and cloud-native volume drivers (AWS EBS, Azure Disk) to scale these concepts to multi-host production clusters.